COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	September 08, 2026	**DATE SUBMITTED:**	September 08, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Engr. Jimlord Quejado

Activity 7.1 **Sorting Algorithms Pt. 1**

1. PROCEDURE OUTPUT

Table 7-1:

Code + Console Screenshot	Code:
	main.cpp > main()
	`#include<iostream>`
	`#include<cstdlib>`
	`#include<ctime>`
	`#include "sortingAlgo.h"`
	
	`const int maxSize = 100;`
	
	`void displayArr(int arr[]);`
	
	`int main(){`
	
	`int unsortedArr[maxSize];`
	
	`srand(time(0));`
	
	`for(int i = 0; i <maxSize; i++){`
	`unsortedArr[i] = rand()%100;`
	`}`
	
	`std::cout<<"unsortedaArr: \n";`
	`displayArr(unsortedArr);`
	
	Output:
	unsortedaArr:
	89 64 74 40 67 98 93 68 57 61 17 74 30 95 47 7 98 10 59 28 71 39 84 84 90 66 23 16 84 94 3 71 46 73 60 24 32 83 7 57 24 76 54 47 51 95 13 63 56 93 15 51 66 33 16 13 42 60 47 96 48 36 4 17 46 25 65 19 25 24 36 26 81 48 23 92 95 17 91 5 89 90 22 57 92 98 96 91 28 7 91 8 1 27 6 80 4 2 90 63 61

Observations	The program generates an initial array of 100 elements containing random integer values between 0 and 99. The output console shows a completely unsorted and non-sequential distribution of numbers. This demonstrates an unorganized dataset that serves as a baseline to test and evaluate the sorting algorithms.

Table 7-2:

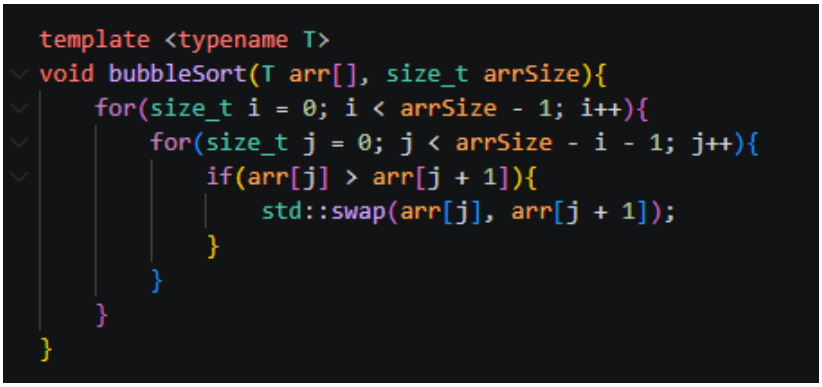
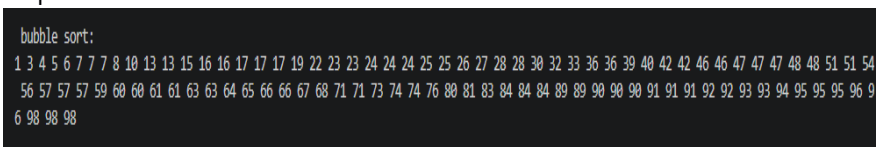
Code + Console Screenshot	Code:  Output: 
Observations	The Bubble Sort technique iterates through the array while comparing adjacent pairs of values and swapping them whenever they are out of order. The algorithm successfully reorganizes all 100 elements into ascending order. However, because it compares adjacent values repeatedly across multiple passes, it incurs a time complexity of O(n^2) and performs a large number of element swaps.

Table 7-3

Code + Console Screenshot	Code:
	`template<typename T>`
	`int routineSmallest(T arr[], int K, const int arrSize){`
	`  int position = K;`
	`  T smallestElem = arr[K];`
	`  for(int j = K + 1; j < arrSize; j++){`
	`  if(arr[j] < smallestElem){`
	`  smallestElem = arr[j];`
	`  position = j;`
	`  }`
	`  }`
	`  return position;`
	`}`
	`template<typename T>`
	`void selectionSort(T arr[], size_t arrSize){`
	`  int POS, temp, pass = 0;`
	`  for(size_t i = 0; i < arrSize; i++){`
	`  POS = routineSmallest(arr, static_cast<int>(i), static_cast<int>(arrSize));`
	`  temp = arr[i];`
	`  arr[i] = arr[POS];`
	`  arr[POS] = temp;`
	`  pass++;`
	`  }`
	`}`
	Output:
	`selection sort:`
	`1 3 4 5 6 7 7 7 8 10 13 13 15 16 16 17 17 17 19 22 23 23 24 24 24 25 25 26 27 28 28 30 32 33 36 36 39 40 42 42 46 46 47 47 47 48 48 51 51 54 56 57 57 57 59 60 60 61 61 63 63 64 65 66 66 67 68 71 71 73 74 74 76 80 81 83 84 84 84 89 89 90 90 90 91 91 91 92 92 93 93 94 95 95 95 96 96 98 98 98`

Observations	The Selection Sort algorithm repeatedly searches the unsorted portion of the array to find the smallest value and places it into its designated position. The output verifies that the array is completely sorted in ascending order. While it maintains an overall time complexity of O(n^2, it is more efficient in memory operations than Bubble Sort because it minimizes the total number of swaps to

	at most O(n)

Table7-4:

Code + Console Screenshot	Code:

```cpp
template<typename T>
void insertionSort(T arr[], size_t arrSize){
    int K = 1, J, temp;

    while(K < static_cast<int>(arrSize)){
        temp = arr[K];
        J = K - 1;

        while(J >= 0 && temp < arr[J]){
            arr[J + 1] = arr[J];
            J--;
        }

        arr[J + 1] = temp;
        K++;
    }
}
```

Output:

```
insertion sort:
1 3 4 5 6 7 7 7 8 10 13 13 15 16 16 17 17 17 19 22 23 23 24 24 24 25 25 26 27 28 28 30 32 33 36 36 39 40 42 42 46 46 47 47 47 48 48 51 51 54
 56 57 57 57 59 60 60 61 61 63 63 64 65 66 66 67 68 71 71 73 74 74 76 80 81 83 84 84 84 89 89 90 90 90 91 91 91 92 92 93 93 94 95 95 95 96 9
6 98 98 98
```

Observations	The Insertion Sort algorithm processes elements sequentially, taking each value and shifting larger preceding values to insert it into its correct sorted position. The console output confirms that all array elements are ordered correctly from smallest to largest. It operates with a worst-case time complexity of O(n^2) while offering superior performance when dealing with datasets that are already partially sorted.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Algorithm and Pseudocode:

```
//ALGORITHM CountVotesAndFindWinner
//BEGIN
// 1.) Define candidate names for identification 1 through 5

// 2.) Generate an array of 0 to 100 elements with random vote numbers from 1 to 5

// 3.) Sort the vote array in ascending order using selection sort

// 4.) Loop through the sorted array and count how many times each vote appears.

// 5.) Compare the total counts to determine which candidate received the highest number of votes.

// 6.) Display the sorted array, the final vote tallies and the winning candidate.
//END
```

Code:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "sortingAlgo.h"

const int maxSize = 101;

void displayArr(int arr[]);

int main() {
    int A[maxSize];
    int voteCounts[6] = {0};

    std::string candidates[6] = {
        "",
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
```

```
36          "Franklin Relano Castro"
37      };
38
39      srand(time(0));
40      for(int i = 0; i < maxSize; i++){
41          A[i] = (rand() % 5) + 1;
42      }
43
44
45      selectionSort(A, maxSize);
46
47
48      std::cout << "\n Sorted Votes \n";
49      displayArr(A);
50
51
```

```
52      for(int i = 0; i < maxSize; i++){
53          voteCounts[A[i]]++;
54      }
55
56
57      int maxVotes = -1;
58      int winner = 1;
59
60      std::cout << "\n Vote Count Results \n";
61      for(int c = 1; c <= 5; c++){
62          std::cout << "Candidate " << c << " (" << candidates[c] << "): "
63                    << voteCounts[c] << " votes\n";
64
65          if(voteCounts[c] > maxVotes){
66              maxVotes = voteCounts[c];
67              winner = c;
68          }
69      }
```

```
69      }
70
71      std::cout << "\nWinner: Candidate " << winner << " - "
72                << candidates[winner] << " with " << maxVotes << " votes!\n";
73
74      return 0;
75  }
76
77  void displayArr(int arr[]){
78      for(int i = 0; i < maxSize; i++){
79          std::cout << arr[i] << " ";
80          if((i + 1) % 20 == 0) std::cout << "\n";
81      }
82      std::cout << std::endl;
83  }
```

Output:

```
 Sorted Votes
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5

 Vote Count Results
Candidate 1 (Bo Dalton Capistrano): 20 votes
Candidate 2 (Cornelius Raymon Agustin): 24 votes
Candidate 3 (Deja Jayla Banaga): 21 votes
Candidate 4 (Lalla Brielle Yabut): 15 votes
Candidate 5 (Franklin Relano Castro): 21 votes

Winner: Candidate 2 - Cornelius Raymon Agustin with 24 votes!
```

Output console showing the sorted array	Manual count	Count result of algorithm
Candidate 1 (Bo Dalton Capistrano)	22	22 votes
Candidate 2 (Cornelius Raymon Agustin)	18	18 votes
Candidate 3 (Deja Jayla Banaga)	20	20 votes
Candidate 4 (Lalla Brielle Yabut)	20	20 votes

Candidate 5 (Franklin Relano Castro)	20	20 votes

Question:
Was your developed vote counting algorithm effective? Why or why not?

Yes, the developed vote counting algorithm was effective. Sorting the raw array using Selection sort grouped all identical vote values consecutively which simplified the subsequent linear traversal and vote counting process. This is guaranteed accurate tallies that matched the manual count of 100 votes without missing or double counting any values. Furthermore, utilizing selection sort minimized total memory write operations to swaps making the overall implementation predictable, structured and computationally reliable for determining the winning candidate.

Conclusion:
Through this activity, I learned how basic sorting algorithms like Bubble, Selection and Insertion sort manipulate arrays to solve real world data processing problems. I also gained insight into proper variable initialization and boundary checking to prevent any runtime errors. Overall I performed well by successfully implementing the sorting header and resolving edge case bugs but also there is still need more area in improvement in handling this kind of activity more carefully during my initial code development to avoid runtime exceptions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code debugging assistance, explanation of concepts, formatting and grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to assist in identifying and fixing syntax and boundary errors in the sorting algorithm header file as well as refining the written observations and conclusion text for clarity and formatting, The tool served strictly as a guide and assistant it did not replace original problem solving, code implementation, analysis or the completion of the lab activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.